Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет «МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчёт по лабораторной работе № 2

по дисциплине «Технологии программирования»

на тему «Основы ООП»

Выполнил:
студент группы БИВТ-25-4-7
Иванов Антон Сергеевич

Проверил:
ассистент
Гласов А. В.

Москва, 2026

Цель и содержание работы

Цель работы — закрепить основы объектно-ориентированного программирования на примере классов для пошаговой компьютерной стратегии. По заданию требовалось реализовать предложенную иерархию, определить поведение игровых объектов и загрузить исходный код в удалённый репозиторий.

Для выполнения работы использовал Python и PyCharm. Классы разместил в отдельных файлах. Названия методов оставил такими, как они даны в задании.

Ход выполнения работы

1. Базовый класс GameObject

Сначала создал класс GameObject. В нём находятся общие для игровых объектов данные: идентификатор, имя и координаты x и y. Значения задаются при создании объекта через __init__(), а методы getId(), getName(), getX() и getY() возвращают соответствующие поля. Поля начинаются с подчёркивания, чтобы было понятно, что менять их напрямую снаружи класса не предполагается.

game_object.py

```python
class GameObject:
    def __init__(self, object_id: int, name: str, x: int, y: int):
        self._id = object_id
        self._name = name
        self._x = x
        self._y = y

    def getID(self) -> int:
        return self._id

    def getName(self) -> str:
        return self._name

    def getX(self) -> int:
        return self._x

    def getY(self) -> int:
        return self._y
```

Рисунок 1 — Базовый класс GameObject

2. Классы Unit и Building

Класс Unit наследуется от GameObject и добавляет запас здоровья _hp. Вызов super().__init__() позволяет не дублировать инициализацию имени, идентификатора и координат. Метод isAlive() проверяет, больше ли здоровье нуля, getHp() возвращает его текущее значение, а receiveDamage() уменьшает здоровье при получении урона. Для отрицательного урона предусмотрена проверка.



Рисунок 2 — Реализация класса Unit

Класс Building тоже наследуется от GameObject, но вместо здоровья хранит состояние постройки. Метод isBuilt() показывает, завершено ли строительство, а метод build() переводит постройку в готовое состояние. Оба класса используют общие методы GameObject, но дополняют их своими свойствами.

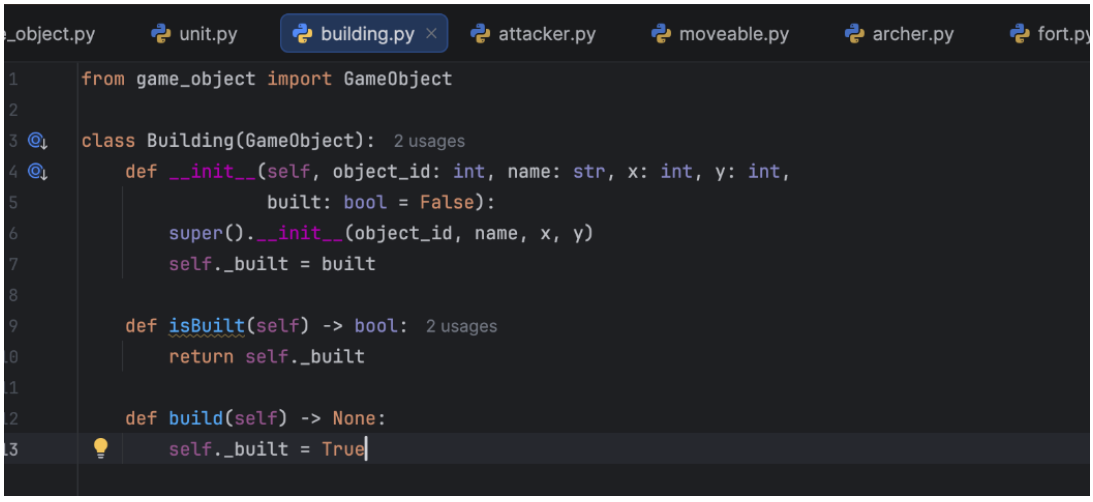


Рисунок 3 — Класс Building с методом build()

3. Интерфейсы Attacker и Moveable

Для обозначения способностей игровых объектов сделал два абстрактных базовых класса на основе ABC. У Attacker объявлен обязательный метод attack(unit), у Moveable — move(x, y). Они содержат только сигнатуры методов с декоратором @abstractmethod. Конкретную реализацию добавляют классы, которые используют эти интерфейсы.

```python
from abc import ABC, abstractmethod
from unit import Unit

class Attacker(ABC):
    @abstractmethod
    def attack(self, unit: Unit) -> None:
        pass
```

Рисунок 4 — Интерфейс Attacker

```python
from abc import ABC, abstractmethod

class Moveable(ABC):
    @abstractmethod
    def move(self, x: int, y: int) -> None:
        pass
```

Рисунок 5 — Интерфейс Moveable

Такое разделение удобно: атаковать может и юнит, и постройка, хотя они принадлежат к разным ветвям иерархии. Само наличие способности не зависит от родительского класса.

4. Класс Archer

Archer наследуется от Unit и реализует сразу два интерфейса: Attacker и Moveable. Помимо здоровья у лучника есть величина урона _damage. Метод attack() вызывает receiveDamage() у выбранного юнита, если оба юнита живы. Метод move() меняет координаты лучника, если он ещё жив. Здесь используется множественное наследование Python.

```python
from unit import Unit
from attacker import Attacker
from moveable import Moveable


class Archer(Unit, Attacker, Moveable):
    def __init__(self, object_id: int, name: str, x: int, y: int,
                 hp: float | int, damage: float | int):
        super().__init__(object_id, name, x, y, hp)
        self._damage = max(0.0, damage)

    def attack(self, unit: Unit) -> None:
        if self.isAlive() and unit.isAlive():
            unit.receiveDamage(self._damage)

    def move(self, x: int, y: int) -> None:
        if self.isAlive():
            self._x = x
            self._y = y
```

Рисунок 6 — Класс Archer: атака и перемещение

5. Класс Fort

Fort наследуется от Building и реализует интерфейс Attacker. Крепость наносит урон указанному юниту через его метод receiveDamage(), но только после завершения строительства и при условии, что цель жива. В отличие от лучника, крепость не реализует Moveable. У Archer и Fort есть общий метод attack(), хотя устроены эти классы по-разному — это пример полиморфизма.



Рисунок 7 — Класс Fort

6. Класс MobileHouse

MobileHouse наследуется от Building и реализует Moveable. Дом на колёсах может менять координаты, когда он построен. Собственный конструктор здесь не нужен, поэтому используется конструктор Building. Это позволяет не повторять уже написанный код.



Рисунок 8 — Класс MobileHouse

7. Размещение проекта на GitHub

Для хранения лабораторных работ использовал ранее созданный репозиторий TP_Labs. Получил его локальную копию командой git clone, затем в папку Lab_2 скопировал восемь Python-файлов. Папка Lab_1 с первой лабораторной при этом осталась без изменений.

После проверки содержимого выполнил git add Lab_2/ и git status. В индекс попали только файлы второй лабораторной, без настроек PyCharm и служебных каталогов.

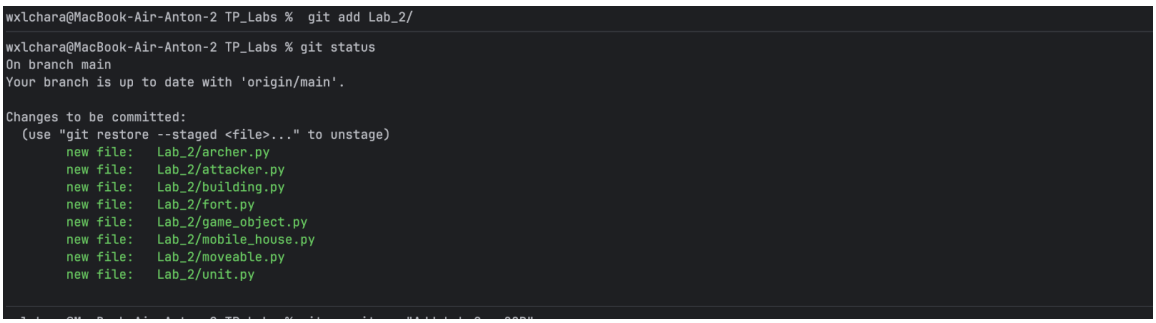


Рисунок 9 — Файлы Lab_2, добавленные в индекс Git

Далее создал коммит командой git commit -m "Add Lab 2 - OOP" и отправил его в ветку main командой git push origin main. Git сообщил об успешной передаче изменений. В репозитории TP_Labs теперь находятся отдельные папки Lab_1 и Lab_2.



Рисунок 10 — Создание коммита и отправка Lab_2 на GitHub

Вывод

В ходе лабораторной работы реализовал иерархию игровых объектов для пошаговой стратегии. Общие свойства вынесены в базовый класс, а отдельные возможности описаны через интерфейсы Attacker и Moveable. На примерах Archer, Fort и MobileHouse показано, как наследование позволяет повторно использовать код, а один интерфейс — применять к разным типам объектов. Готовые исходные файлы добавлены в репозиторий TP_Labs на GitHub.